

Final Project Database LEC



Anggota Kelompok:

Aryadito Galih Akbar Wicaksono - 2802413341

Giovan Ivander - 2802389166

Kautsar Bintang Sinaga – 2802414590

Nathan Alexandro Thiosa - 2802401404

Stephen - 2802396045

Ranzell Syaidina Ouwrilian - 2802398920

1. Nama Perusahaan:

Tokopedia



2. Latar Belakang:

Tokopedia merupakan salah satu platform e-commerce terbesar dan paling berpengaruh di Indonesia yang berperan sebagai penghubung antara penjual dan pembeli dalam ruang digital. Secara sederhana, Tokopedia dapat dipahami sebagai pasar daring (online marketplace) yang memungkinkan individu maupun pelaku usaha dari berbagai latar belakang untuk melakukan aktivitas perdagangan secara elektronik tanpa harus memiliki toko fisik. Model bisnis ini mengubah paradigma masyarakat mengenai kegiatan jual beli yang selama ini bersifat konvensional. Di dalam ekosistem Tokopedia, setiap orang dapat membuka toko, memasarkan produknya, menerima pembayaran secara digital, dan mengatur pengiriman barang hanya dengan bermodalkan perangkat gawai dan koneksi internet. Kemudahan ini membuat Tokopedia menjadi wadah inklusif yang menjangkau seluruh lapisan masyarakat, dari pelaku UMKM di kota kecil hingga konsumen di daerah terpencil yang sebelumnya sulit mengakses produk tertentu.

Dengan jutaan transaksi yang terjadi setiap harinya, Tokopedia menjadi representasi nyata dari sistem informasi modern yang kompleks dan terintegrasi. Dalam konteks ini, penting untuk memahami bagaimana proses transaksi berlangsung di dalam platform tersebut, mulai dari pemesanan, pembayaran, hingga konfirmasi penerimaan barang.

Ruang lingkup yang kami kerjakan dalam proyek ini, kami memfokuskan pembahasan hanya pada proses transaksi, sebagai inti dari interaksi ekonomi digital antara pengguna dan sistem. Untuk memodelkan proses ini secara teknis, akan dilakukan perancangan Entity Relationship Diagram (ERD) dan pembuatan query SQL yang dijalankan melalui Oracle APEX, sebagai salah satu platform berbasis cloud yang mendukung pengembangan aplikasi berbasis data. Dengan pendekatan ini, ruang lingkup project akan dibatasi pada perancangan dan implementasi struktur data serta logika transaksi, tanpa mencakup aspek lain seperti sistem logistik, manajemen iklan, atau layanan pelanggan.

3. Bussiness Process:

Secara umum, business process Tokopedia terbagi menjadi tiga komponen utama: proses untuk *merchant* (penjual), proses untuk *customer* (pembeli), dan proses internal platform sebagai mediator. Ketiga elemen ini saling terintegrasi dalam sistem digital berbasis web dan mobile application yang memfasilitasi seluruh aktivitas transaksi secara real-time dan terotomatisasi. Tokopedia tidak memproduksi barang secara langsung, melainkan bertindak sebagai fasilitator yang menjembatani antara penjual dan pembeli melalui ekosistem yang aman, nyaman, dan efisien. berikut adalah ketiga komponen tersebut:

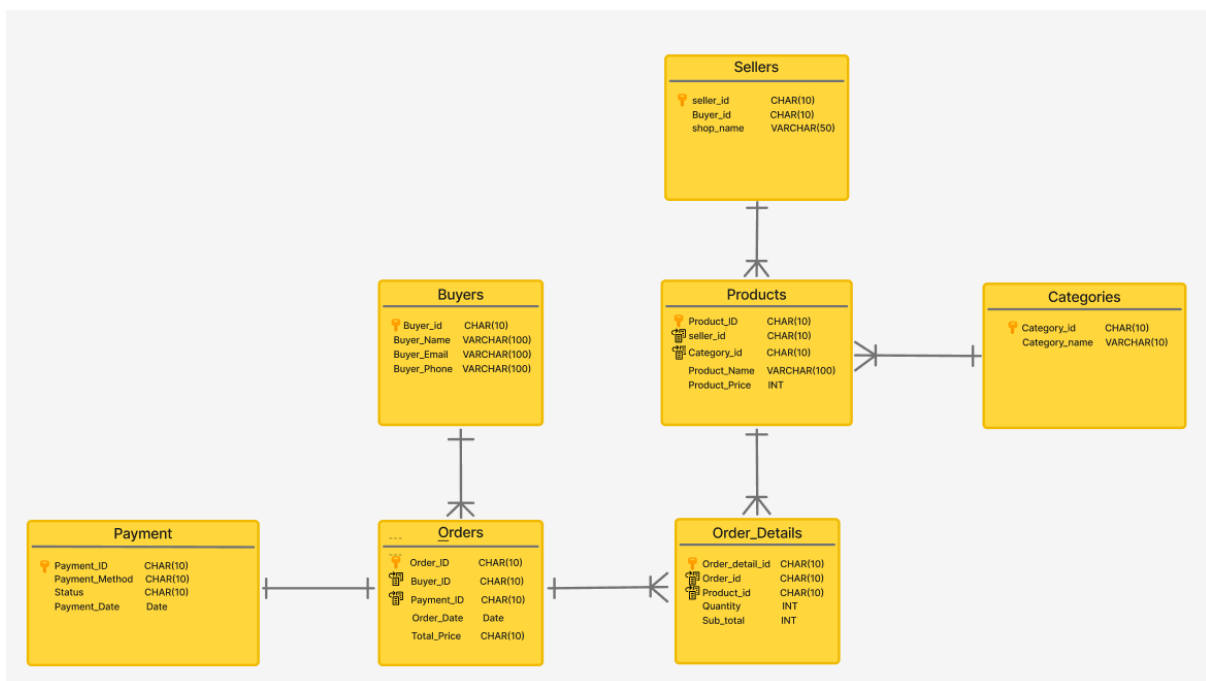
- **Proses untuk Merchant (Penjual):**
 - Registrasi & Pembuatan Toko:
 - Penjual membuat akun Tokopedia melalui website atau aplikasi. Setelah akun aktif, mereka bisa membuka toko dengan memilih nama toko, lokasi, serta mengisi data legal (opsional untuk seller kecil, tapi wajib buat seller besar).
 - Pengunggahan Produk:

- Produk diunggah lengkap dengan nama, kategori, harga, stok, deskripsi, dan foto.
 - Tokopedia menyediakan fitur optimasi SEO internal supaya produk lebih mudah ditemukan di pencarian pengguna.
- Pengelolaan Toko:
 - Toko punya dashboard yang menampilkan data performa penjualan, konversi, traffic, dan pengelolaan stok.
 - Penjual juga bisa mengatur diskon, voucher, dan bundling produk sesuai kebutuhan promosi.
- Fitur Promosi & Iklan:
 - Penjual dapat menggunakan TopAds untuk mempromosikan produk agar tampil di posisi atas.
 - Ada juga fitur “flash sale”, campaign harian, dan kerjasama dengan Tokopedia untuk program diskon masif (misal saat Harbolnas).
- Pemrosesan Pesanan & Pengiriman:
 - Saat ada order, penjual mendapat notifikasi otomatis.
 - Barang dikemas dan dikirim lewat jasa ekspedisi (JNE, Sicepat, AnterAja, dll) yang bisa diintegrasikan langsung.
 - Sistem akan tracking otomatis mulai dari penjual sampai barang diterima pembeli.
- **Proses untuk Customer (Pembeli):**
 - Pencarian & Eksplorasi Produk:
 - Pembeli bisa mencari produk berdasarkan keyword, kategori, filter harga, lokasi, atau rating.
 - Algoritma rekomendasi Tokopedia juga menampilkan produk berdasarkan histori pembelian dan pencarian sebelumnya.
 - Pemesanan Produk:
 - Pembeli memasukkan produk ke keranjang lalu lanjut ke checkout.
 - Bisa beli dari satu toko maupun banyak toko sekaligus (multi-seller checkout).
 - Pilihan Pembayaran:

- Pembeli disediakan berbagai metode: transfer bank, e-wallet (OVO, GoPay), kartu kredit, cicilan, hingga Tokopedia PayLater.
 - Semua transaksi diproses melalui sistem escrow, jadi aman: penjual tidak langsung mendapat uang hingga barang diterima pembeli.
- Penerimaan Barang & Ulasan:
 - Setelah barang sampai, pembeli wajib klik “pesanan diterima” agar dana dicairkan ke penjual.
 - Setelah itu, pembeli bisa kasih rating dan ulasan sebagai feedback buat toko & sistem rekomendasi Tokopedia.
- **Proses Internal Tokopedia (Sebagai Mediator):**
 - Manajemen Transaksi Otomatis:
 - Tokopedia mengelola seluruh alur transaksi secara terintegrasi dari hulu ke hilir.
 - Sistem akan otomatis memverifikasi pembayaran, memvalidasi order, dan menghubungkan penjual dengan ekspedisi.
 - Escrow & Keamanan Transaksi:
 - Dana pembeli ditahan sementara oleh Tokopedia untuk memastikan barang benar-benar diterima.
 - Ini mencegah potensi penipuan dan memberikan rasa aman bagi kedua pihak.
 - Integrasi Ekspedisi & Pelacakan Real-Time.
 - Tokopedia terhubung langsung dengan ekspedisi lewat sistem API.
 - Pembeli dan penjual bisa melacak barang secara real-time lewat aplikasi.
 - Teknologi AI & Data Analytics:
 - Tokopedia menggunakan AI untuk rekomendasi produk, personalisasi tampilan, serta deteksi fraud (penipuan atau manipulasi).
 - Sistem juga menganalisis performa toko, tren konsumen, dan memberikan insight ke penjual.
 - Customer Service & Dispute Resolution:

- Tokopedia menyediakan CS 24/7 yang bisa diakses via chat dan email.
- Jika terjadi masalah dalam transaksi (barang rusak, gak sesuai, hilang), Tokopedia jadi penengah untuk penyelesaian sengketa.

4. ERD



5. CREATE TABLE:

1. Membuat tabel Buyers

```
1 CREATE TABLE Buyers (  
2     Buyer_id CHAR(10) PRIMARY KEY,  
3     Buyer_Name VARCHAR(100),  
4     Buyer_Email VARCHAR(100),  
5     Buyer_Phone VARCHAR(100)  
6 );  
7  
8 DESCRIBE Buyers
```

Results Explain Describe Saved SQL History

Object Type TABLE ? Object BUYERS ?

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
BUYERS	BUYER_ID	CHAR	10	-	-	1	-	-	-
	BUYER_NAME	VARCHAR2	100	-	-	-	✓	-	-
	BUYER_EMAIL	VARCHAR2	100	-	-	-	✓	-	-
	BUYER_PHONE	VARCHAR2	100	-	-	-	✓	-	-

```
CREATE TABLE Buyers (  
    Buyer_id CHAR(10) PRIMARY KEY,  
    Buyer_Name VARCHAR(100),  
    Buyer_Email VARCHAR(100),  
    Buyer_Phone VARCHAR(100)  
);
```

```
DESCRIBE Buyers
```

2. Membuat tabel Sellers

```
9
10 CREATE TABLE Sellers (
11     seller_id CHAR(10) PRIMARY KEY,
12     Buyer_id CHAR(10),
13     shop_name VARCHAR(50),
14     FOREIGN KEY (Buyer_id) REFERENCES Buyers(Buyer_id)
15 );
16
17 DESCRIBE Sellers
18
```

Results Explain **Describe** Saved SQL History

Object Type **TABLE** Object **SELLERS**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
SELLERS	SELLER_ID	CHAR	10	-	-	1	-	-	-
	BUYER_ID	CHAR	10	-	-	-	✓	-	-
	SHOP_NAME	VARCHAR2	50	-	-	-	✓	-	-

```
CREATE TABLE Sellers (
    seller_id CHAR(10) PRIMARY KEY,
    Buyer_id CHAR(10),
    shop_name VARCHAR(50),
    FOREIGN KEY (Buyer_id) REFERENCES Buyers(Buyer_id)
);
```

```
DESCRIBE Sellers
```

3. Membuat tabel Categories

```
19 CREATE TABLE Categories (  
20     Category_id CHAR(10) PRIMARY KEY,  
21     Category_name VARCHAR(10)  
22 );  
23  
24 DESCRIBE Categories  
25
```

Results Explain **Describe** Saved SQL History

Object Type TABLE ? Object CATEGORIES ?

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
CATEGORIES	CATEGORY_ID	CHAR	10	-	-	1	-	-	-
	CATEGORY_NAME	VARCHAR2	10	-	-	-	✓	-	-

```
CREATE TABLE Categories (  
    Category_id CHAR(10) PRIMARY KEY,  
    Category_name VARCHAR(10)  
);
```

```
DESCRIBE Categories
```

4. Membuat tabel Products

```

26 CREATE TABLE Products (
27     Product_id CHAR(10) PRIMARY KEY,
28     seller_id CHAR(10),
29     Category_id CHAR(10),
30     Product_Name VARCHAR(100),
31     Product_Price INT,
32     FOREIGN KEY (seller_id) REFERENCES Sellers(seller_id),
33     FOREIGN KEY (Category_id) REFERENCES Categories(Category_id)
34 );
35
36 DESCRIBE Products
37

```

Results Explain Describe Saved SQL History

Object Type TABLE ? Object PRODUCTS ?

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
PRODUCTS	PRODUCT_ID	CHAR	10	-	-	1	-	-	-
	SELLER_ID	CHAR	10	-	-	-	✓	-	-
	CATEGORY_ID	CHAR	10	-	-	-	✓	-	-
	PRODUCT_NAME	VARCHAR2	100	-	-	-	✓	-	-
	PRODUCT_PRICE	NUMBER	22	-	0	-	✓	-	-

```

CREATE TABLE Products (
    Product_id CHAR(10) PRIMARY KEY,
    seller_id CHAR(10),
    Category_id CHAR(10),
    Product_Name VARCHAR(100),
    Product_Price INT,
    FOREIGN KEY (seller_id) REFERENCES Sellers(seller_id),
    FOREIGN KEY (Category_id) REFERENCES Categories(Category_id)
);

```

DESCRIBE Products

5. Membuat tabel Payment

```
37
38 CREATE TABLE Payment (
39     Payment_ID CHAR(10) PRIMARY KEY,
40     Payment_Method CHAR(10),
41     Status CHAR(10),
42     Payment_Date DATE
43 );
44
45 DESCRIBE Payment
46
```

Results Explain Describe Saved SQL History

Object Type TABLE ? Object PAYMENT ?

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
PAYMENT	PAYMENT_ID	CHAR	10	-	-	1	-	-	-
	PAYMENT_METHOD	CHAR	10	-	-	-	✓	-	-
	STATUS	CHAR	10	-	-	-	✓	-	-
	PAYMENT_DATE	DATE	7	-	-	-	✓	-	-

```
CREATE TABLE Payment (
    Payment_ID CHAR(10) PRIMARY KEY,
    Payment_Method CHAR(10),
    Status CHAR(10),
    Payment_Date DATE
);
```

```
DESCRIBE Payment
```

6. Membuat tabel Orders

```

47 CREATE TABLE Orders (
48     Order_ID CHAR(10) PRIMARY KEY,
49     Buyer_ID CHAR(10),
50     Payment_ID CHAR(10),
51     Order_Date DATE,
52     Total_Price CHAR(10),
53     FOREIGN KEY (Buyer_ID) REFERENCES Buyers(Buyer_id),
54     FOREIGN KEY (Payment_ID) REFERENCES Payment(Payment_ID)
55 );
56
57 DESCRIBE Orders
58

```

Results Explain Describe Saved SQL History

Object Type TABLE ? Object ORDERS ?

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
ORDERS	ORDER_ID	CHAR	10	-	-	1	-	-	-
	BUYER_ID	CHAR	10	-	-	-	✓	-	-
	PAYMENT_ID	CHAR	10	-	-	-	✓	-	-
	ORDER_DATE	DATE	7	-	-	-	✓	-	-
	TOTAL_PRICE	CHAR	10	-	-	-	✓	-	-

```

CREATE TABLE Orders (
    Order_ID CHAR(10) PRIMARY KEY,
    Buyer_ID CHAR(10),
    Payment_ID CHAR(10),
    Order_Date DATE,
    Total_Price CHAR(10),
    FOREIGN KEY (Buyer_ID) REFERENCES Buyers(Buyer_id),
    FOREIGN KEY (Payment_ID) REFERENCES Payment(Payment_ID)
);

```

```
DESCRIBE Orders
```

7. Membuat tabel OrderDetails

```
58
59 CREATE TABLE Order_Details (
60     Order_detail_id CHAR(10) PRIMARY KEY,
61     Order_id CHAR(10),
62     Product_id CHAR(10),
63     Quantity INT,
64     Sub_total INT,
65     FOREIGN KEY (Order_id) REFERENCES Orders(Order_ID),
66     FOREIGN KEY (Product_id) REFERENCES Products(Product_id)
67 );
68
69 DESCRIBE Order_Details
70
```

Results Explain Describe Saved SQL History

Object Type TABLE ? Object ORDERS ?

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
ORDERS	ORDER_ID	CHAR	10	-	-	1	-	-	-
	BUYER_ID	CHAR	10	-	-	-	✓	-	-
	PAYMENT_ID	CHAR	10	-	-	-	✓	-	-
	ORDER_DATE	DATE	7	-	-	-	✓	-	-
	TOTAL_PRICE	CHAR	10	-	-	-	✓	-	-

```
CREATE TABLE Order_Details (
    Order_detail_id CHAR(10) PRIMARY KEY,
    Order_id CHAR(10),
    Product_id CHAR(10),
    Quantity INT,
    Sub_total INT,
    FOREIGN KEY (Order_id) REFERENCES Orders(Order_ID),
    FOREIGN KEY (Product_id) REFERENCES Products(Product_id)
);
```

```
DESCRIBE Order_Details
```

INSERT DATA:

1. INSERT DATA ke Buyers

```
70
71 INSERT ALL
72 INTO Buyers VALUES ('B001', 'Buyer1', 'buyer1@example.com', '08854751070')
73 INTO Buyers VALUES ('B002', 'Buyer2', 'buyer2@example.com', '08130035916')
74 INTO Buyers VALUES ('B003', 'Buyer3', 'buyer3@example.com', '08280483383')
75 INTO Buyers VALUES ('B004', 'Buyer4', 'buyer4@example.com', '08396387730')
76 INTO Buyers VALUES ('B005', 'Buyer5', 'buyer5@example.com', '08390971736')
77 INTO Buyers VALUES ('B006', 'Buyer6', 'buyer6@example.com', '08259267973')
78 INTO Buyers VALUES ('B007', 'Buyer7', 'buyer7@example.com', '08133518642')
79 INTO Buyers VALUES ('B008', 'Buyer8', 'buyer8@example.com', '08568530298')
80 INTO Buyers VALUES ('B009', 'Buyer9', 'buyer9@example.com', '08441251826')
81 INTO Buyers VALUES ('B010', 'Buyer10', 'buyer10@example.com', '08601326050')
82 SELECT * FROM dual;
83
84 SELECT * FROM Buyers
85
86
```

Results			
Explain Describe Saved SQL History			
BUYER_ID	BUYER_NAME	BUYER_EMAIL	BUYER_PHONE
B001	Buyer1	buyer1@example.com	08854751070
B002	Buyer2	buyer2@example.com	08130035916
B003	Buyer3	buyer3@example.com	08280483383
B004	Buyer4	buyer4@example.com	08396387730
B005	UpdatedBuyer5	buyer5@example.com	08390971736
B006	Buyer6	buyer6@example.com	08259267973
B007	Buyer7	buyer7@example.com	08133518642
B008	Buyer8	buyer8@example.com	08568530298
B009	Buyer9	buyer9@example.com	08441251826
B010	Buyer10	buyer10@example.com	08601326050
10 rows returned in 0.02 seconds Download			

2. INSERT DATA ke Sellers

```
86 INSERT ALL
87 INTO Sellers VALUES ('S001', 'B001', 'Shop1')
88 INTO Sellers VALUES ('S002', 'B002', 'Shop2')
89 INTO Sellers VALUES ('S003', 'B003', 'Shop3')
90 INTO Sellers VALUES ('S004', 'B004', 'Shop4')
91 INTO Sellers VALUES ('S005', 'B005', 'Shop5')
92 INTO Sellers VALUES ('S006', 'B006', 'Shop6')
93 INTO Sellers VALUES ('S007', 'B007', 'Shop7')
94 INTO Sellers VALUES ('S008', 'B008', 'Shop8')
95 INTO Sellers VALUES ('S009', 'B009', 'Shop9')
96 INTO Sellers VALUES ('S010', 'B010', 'Shop10')
97 SELECT * FROM dual;
98
99 SELECT * FROM Sellers
100
```

Results		
Explain Describe Saved SQL History		
SELLER_ID	BUYER_ID	SHOP_NAME
S001	B001	Shop1
S002	B002	Shop2
S003	B003	Shop3
S004	B004	Shop4
S005	B005	Shop5
S006	B006	Shop6
S007	B007	Shop7
S008	B008	Shop8
S009	B009	Shop9
S010	B010	Shop10

3. INSERT DATA ke Categories

```
100
101 INSERT ALL
102 INTO Categories VALUES ('C001', 'Vegetables')
103 INTO Categories VALUES ('C002', 'Fruits')
104 INTO Categories VALUES ('C003', 'Dairy')
105 INTO Categories VALUES ('C004', 'Meat')
106 INTO Categories VALUES ('C005', 'Grains')
107 INTO Categories VALUES ('C006', 'Herbs')
108 INTO Categories VALUES ('C007', 'Spices')
109 INTO Categories VALUES ('C008', 'Seeds')
110 INTO Categories VALUES ('C009', 'Fish')
111 INTO Categories VALUES ('C010', 'Beverages')
112 SELECT * FROM dual;
113
114 SELECT * FROM Categories
```

Results Explain Describe Saved SQL History

CATEGORY_ID	CATEGORY_NAME
C001	Vegetables
C002	Fruits
C003	Dairy
C004	Meat
C005	Grains
C006	Herbs
C007	Spices
C008	Seeds
C009	Fish
C010	Beverages

4. INSERT DATA ke Products

```
116 INSERT ALL
117 INTO Products VALUES ('P001', 'S001', 'C001', 'Carrot', 10)
118 INTO Products VALUES ('P002', 'S002', 'C002', 'Apple', 15)
119 INTO Products VALUES ('P003', 'S003', 'C003', 'Milk', 20)
120 INTO Products VALUES ('P004', 'S004', 'C004', 'Chicken', 30)
121 INTO Products VALUES ('P005', 'S005', 'C005', 'Rice', 12)
122 INTO Products VALUES ('P006', 'S006', 'C006', 'Basil', 8)
123 INTO Products VALUES ('P007', 'S007', 'C007', 'Turmeric', 11)
124 INTO Products VALUES ('P008', 'S008', 'C008', 'Sunflower Seeds', 13)
125 INTO Products VALUES ('P009', 'S009', 'C009', 'Tilapia', 25)
126 INTO Products VALUES ('P010', 'S010', 'C010', 'Honey Tea', 18)
127 SELECT * FROM dual;
128
129 SELECT * FROM Products
130
```

Results Explain Describe Saved SQL History

PRODUCT_ID	SELLER_ID	CATEGORY_ID	PRODUCT_NAME	PRODUCT_PRICE
P001	S001	C001	Carrot	10
P002	S002	C002	Apple	15
P003	S003	C003	Milk	20
P004	S004	C004	Chicken	99
P005	S005	C005	Rice	12
P006	S006	C006	Basil	8
P007	S007	C007	Turmeric	11
P008	S008	C008	Sunflower Seeds	13
P009	S009	C009	Tilapia	25
P010	S010	C010	Honey Tea	18

5. INSERT DATA ke Payment

```
131 INSERT ALL
132 INTO Payment VALUES ('PMT001', 'Cash', 'Completed', DATE '2024-05-01')
133 INTO Payment VALUES ('PMT002', 'Transfer', 'Completed', DATE '2024-05-02')
134 INTO Payment VALUES ('PMT003', 'Credit', 'Pending', DATE '2024-05-03')
135 INTO Payment VALUES ('PMT004', 'Cash', 'Completed', DATE '2024-05-04')
136 INTO Payment VALUES ('PMT005', 'Transfer', 'Completed', DATE '2024-05-05')
137 INTO Payment VALUES ('PMT006', 'Credit', 'Failed', DATE '2024-05-06')
138 INTO Payment VALUES ('PMT007', 'Cash', 'Completed', DATE '2024-05-07')
139 INTO Payment VALUES ('PMT008', 'Transfer', 'Pending', DATE '2024-05-08')
140 INTO Payment VALUES ('PMT009', 'Credit', 'Completed', DATE '2024-05-09')
141 INTO Payment VALUES ('PMT010', 'Cash', 'Completed', DATE '2024-05-10')
142 SELECT * FROM dual;
143
144 SELECT * FROM Payment
```

PAYMENT_ID	PAYMENT_METHOD	STATUS	PAYMENT_DATE
PMT001	Cash	Completed	5/1/2024
PMT002	Transfer	Completed	5/2/2024
PMT003	Credit	Pending	5/3/2024
PMT004	Cash	Completed	5/4/2024
PMT005	Transfer	Completed	5/5/2024
PMT006	Credit	Failed	5/6/2024
PMT007	Cash	Completed	5/7/2024
PMT008	Transfer	Pending	5/8/2024
PMT009	Credit	Completed	5/9/2024
PMT010	Cash	Completed	5/10/2024

6. INSERT DATA Ke Orders

```
145
146 INSERT ALL
147 INTO Orders VALUES ('O001', 'B001', 'PMT001', DATE '2024-05-01', '100')
148 INTO Orders VALUES ('O002', 'B002', 'PMT002', DATE '2024-05-02', '200')
149 INTO Orders VALUES ('O003', 'B003', 'PMT003', DATE '2024-05-03', '150')
150 INTO Orders VALUES ('O004', 'B004', 'PMT004', DATE '2024-05-04', '220')
151 INTO Orders VALUES ('O005', 'B005', 'PMT005', DATE '2024-05-05', '80')
152 INTO Orders VALUES ('O006', 'B006', 'PMT006', DATE '2024-05-06', '175')
153 INTO Orders VALUES ('O007', 'B007', 'PMT007', DATE '2024-05-07', '90')
154 INTO Orders VALUES ('O008', 'B008', 'PMT008', DATE '2024-05-08', '110')
155 INTO Orders VALUES ('O009', 'B009', 'PMT009', DATE '2024-05-09', '130')
156 INTO Orders VALUES ('O010', 'B010', 'PMT010', DATE '2024-05-10', '140')
157 SELECT * FROM dual;
158
159 SELECT * FROM Orders
```

ORDER_ID	BUYER_ID	PAYMENT_ID	ORDER_DATE	TOTAL_PRICE
O001	B001	PMT001	5/1/2024	100
O002	B002	PMT002	5/2/2024	200
O003	B003	PMT003	5/3/2024	150
O004	B004	PMT004	5/4/2024	220
O005	B005	PMT005	5/5/2024	80
O006	B006	PMT006	5/6/2024	175
O007	B007	PMT007	5/7/2024	90
O008	B008	PMT008	5/8/2024	110
O009	B009	PMT009	5/9/2024	130
O010	B010	PMT010	5/10/2024	140

7. INSERT DATA ke OrderDetails

```
161 INSERT ALL
162 INTO Order_Details VALUES ('OD001', 'O001', 'P001', 2, 20)
163 INTO Order_Details VALUES ('OD002', 'O002', 'P002', 3, 45)
164 INTO Order_Details VALUES ('OD003', 'O003', 'P003', 1, 20)
165 INTO Order_Details VALUES ('OD004', 'O004', 'P004', 2, 60)
166 INTO Order_Details VALUES ('OD005', 'O005', 'P005', 4, 48)
167 INTO Order_Details VALUES ('OD006', 'O006', 'P006', 5, 40)
168 INTO Order_Details VALUES ('OD007', 'O007', 'P007', 2, 22)
169 INTO Order_Details VALUES ('OD008', 'O008', 'P008', 3, 39)
170 INTO Order_Details VALUES ('OD009', 'O009', 'P009', 1, 25)
171 INTO Order_Details VALUES ('OD010', 'O010', 'P010', 2, 36)
172 SELECT * FROM dual;
173
174 SELECT * FROM Order_Details
175
```

Results Explain Describe Saved SQL History

ORDER_DETAIL_ID	ORDER_ID	PRODUCT_ID	QUANTITY	SUB_TOTAL
OD001	O001	P001	2	20
OD002	O002	P002	3	45
OD003	O003	P003	1	20
OD004	O004	P004	2	60
OD005	O005	P005	4	48
OD006	O006	P006	5	40
OD007	O007	P007	2	22
OD008	O008	P008	3	39
OD009	O009	P009	1	25
OD010	O010	P010	2	36

UPDATE DATA:

1. UPDATE DATA ke Buyers

```
176 -- update buyers
177 UPDATE Buyers
178 SET Buyer_Name = 'UpdatedBuyer5'
179 WHERE Buyer_id = 'B005';
180
181 SELECT * FROM Buyers
182
```

BUYER_ID	BUYER_NAME	BUYER_EMAIL	BUYER_PHONE
B001	Buyer1	buyer1@example.com	08854751070
B002	Buyer2	buyer2@example.com	08130035916
B003	Buyer3	buyer3@example.com	08280483383
B004	Buyer4	buyer4@example.com	08396387730
B005	UpdatedBuyer5	buyer5@example.com	08390971736
B006	Buyer6	buyer6@example.com	08259267973
B007	Buyer7	buyer7@example.com	08133518642
B008	Buyer8	buyer8@example.com	08568530298
B009	Buyer9	buyer9@example.com	08441251826
B010	Buyer10	buyer10@example.com	08601326050

10 rows returned in 0.00 seconds [Download](#)

UPDATE Buyers

SET Buyer_Name = 'UpdatedBuyer5'

WHERE Buyer_id = 'B005';

SELECT * FROM Buyers

2. UPDATE DATA ke Products

```
183 -- update products
184 UPDATE Products
185 SET Product_Price = 99
186 WHERE Product_id = 'P004';
187
188 SELECT * FROM Products
189
```

PRODUCT_ID	SELLER_ID	CATEGORY_ID	PRODUCT_NAME	PRODUCT_PRICE
P001	S001	C001	Carrot	10
P002	S002	C002	Apple	15
P003	S003	C003	Milk	20
P004	S004	C004	Chicken	99
P005	S005	C005	Rice	12
P006	S006	C006	Basil	8
P007	S007	C007	Turmeric	11
P008	S008	C008	Sunflower Seeds	13
P009	S009	C009	Tilapia	25
P010	S010	C010	Honey Tea	18

10 rows returned in 0.02 seconds [Download](#)

UPDATE Products

SET Product_Price = 99

WHERE Product_id = 'P004';

SELECT * FROM Products

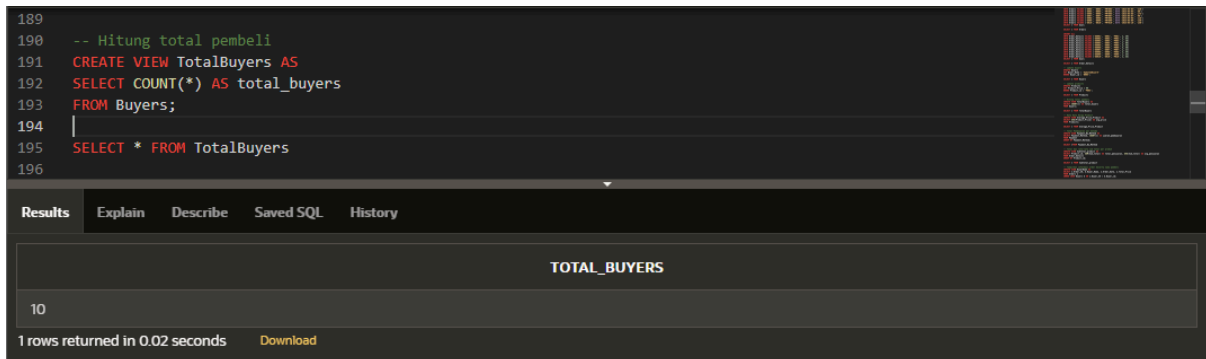
DDL CREATE VIEW:

1. GROUP FUNCTION

- a. CREATE VIEW TotalBuyers AS
SELECT COUNT(*) AS total_buyers
FROM Buyers;

SELECT * FROM TotalBuyers

```
189
190 -- Hitung total pembeli
191 CREATE VIEW TotalBuyers AS
192 SELECT COUNT(*) AS total_buyers
193 FROM Buyers;
194
195 SELECT * FROM TotalBuyers
196
```



The screenshot shows a SQL IDE interface. The top pane contains the SQL code for creating the 'TotalBuyers' view and querying it. The bottom pane shows the results of the query. The results table has a single column named 'TOTAL_BUYERS' and one row with the value '10'. Below the table, it states '1 rows returned in 0.02 seconds' and provides a 'Download' link.

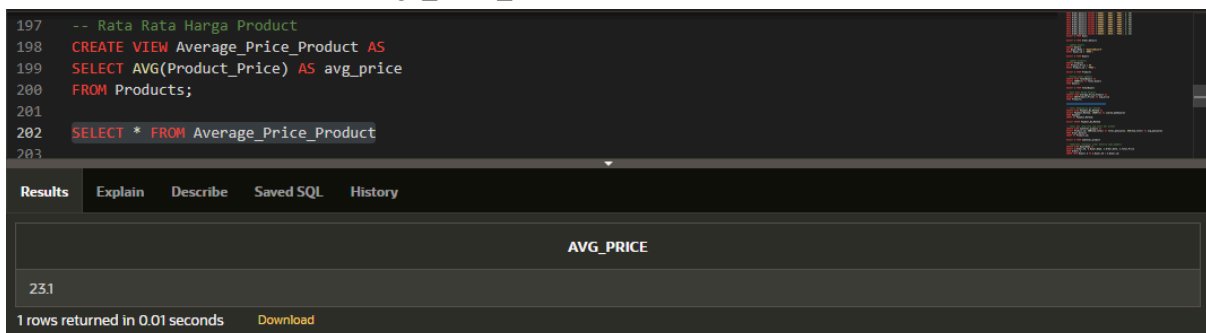
TOTAL_BUYERS
10

1 rows returned in 0.02 seconds [Download](#)

- b. CREATE VIEW Average_Price_Product AS
SELECT AVG(Product_Price) AS avg_price
FROM Products;

SELECT * FROM Average_Price_Product

```
197 -- Rata Rata Harga Product
198 CREATE VIEW Average_Price_Product AS
199 SELECT AVG(Product_Price) AS avg_price
200 FROM Products;
201
202 SELECT * FROM Average_Price_Product
203
```



The screenshot shows a SQL IDE interface. The top pane contains the SQL code for creating the 'Average_Price_Product' view and querying it. The bottom pane shows the results of the query. The results table has a single column named 'AVG_PRICE' and one row with the value '231'. Below the table, it states '1 rows returned in 0.01 seconds' and provides a 'Download' link.

AVG_PRICE
231

1 rows returned in 0.01 seconds [Download](#)

- c. CREATE VIEW Payment_By_Method AS
SELECT Payment_Method, COUNT(*) AS jumlah_pembayaran
FROM Payment
GROUP BY Payment_Method;

SELECT * FROM Payment_By_Method

```
204 -- Total Pembayaran per metode
205 CREATE VIEW Payment_By_Method AS
206 SELECT Payment_Method, COUNT(*) AS jumlah_pembayaran
207 FROM Payment
208 GROUP BY Payment_Method;
209
210 SELECT * FROM Payment_By_Method
211
```

PAYMENT_METHOD	JUMLAH PEMBAYARAN
Cash	4
Transfer	3
Credit	3

3 rows returned in 0.01 seconds [Download](#)

- d. CREATE VIEW Subtotal_product AS
SELECT Product_id, SUM(Sub_total) AS total_penjualan, AVG(Sub_total) AS
avg_penjualan
FROM Order_Details
GROUP BY Product_id;

SELECT * FROM Subtotal_product

```
212 -- Total dan rata-rata sub total per produk
213 CREATE VIEW Subtotal_product AS
214 SELECT Product_id, SUM(Sub_total) AS total_penjualan, AVG(Sub_total) AS avg_penjualan
215 FROM Order_Details
216 GROUP BY Product_id;
217
218 SELECT * FROM Subtotal_product
219
```

PRODUCT_ID	TOTAL PENJUALAN	AVG PENJUALAN
P006	40	40
P007	22	22
P003	20	20
P004	60	60
P005	48	48
P008	39	39
P010	36	36
P001	20	20
P009	25	25
P002	45	45

10 rows returned in 0.01 seconds [Download](#)

2. INNER JOIN

- a. CREATE VIEW BuyerName AS
SELECT o.Order_ID, b.Buyer_Name, o.Order_Date, o.Total_Price
FROM Orders o
INNER JOIN Buyers b ON o.Buyer_ID = b.Buyer_id;

SELECT * FROM BuyerName

```
220 -- Tampilkan informasi order beserta nama pembeli
221 CREATE VIEW BuyerName AS
222 SELECT o.Order_ID, b.Buyer_Name, o.Order_Date, o.Total_Price
223 FROM Orders o
224 INNER JOIN Buyers b ON o.Buyer_ID = b.Buyer_id;
225
226 SELECT * FROM BuyerName
```

ORDER_ID	BUYER_NAME	ORDER_DATE	TOTAL_PRICE
O001	Buyer1	5/1/2024	100
O002	Buyer2	5/2/2024	200
O003	Buyer3	5/3/2024	150
O004	Buyer4	5/4/2024	220
O005	UpdatedBuyer5	5/5/2024	80
O006	Buyer6	5/6/2024	175
O007	Buyer7	5/7/2024	90
O008	Buyer8	5/8/2024	110
O009	Buyer9	5/9/2024	130
O010	Buyer10	5/10/2024	140

10 rows returned in 0.01 seconds [Download](#)

- b. CREATE VIEW ProductNameJoin AS
SELECT p.Product_Name, c.Category_name, p.Product_Price
FROM Products p
INNER JOIN Categories c ON p.Category_id = c.Category_id;

SELECT * FROM ProductNameJoin

```
228 -- Tampilkan detail produk dan kategorinya
229 CREATE VIEW ProductNameJoin AS
230 SELECT p.Product_Name, c.Category_name, p.Product_Price
231 FROM Products p
232 INNER JOIN Categories c ON p.Category_id = c.Category_id;
233
234 SELECT * FROM ProductNameJoin
```

PRODUCT_NAME	CATEGORY_NAME	PRODUCT_PRICE
Carrot	Vegetables	10
Apple	Fruits	15
Milk	Dairy	20
Chicken	Meat	99
Rice	Grains	12
Basil	Herbs	8
Turmeric	Spices	11
Sunflower Seeds	Seeds	13
Tilapia	Fish	25
Honey Tea	Beverages	18

10 rows returned in 0.02 seconds [Download](#)

3. OUTER JOIN

- a. CREATE VIEW BuyerWithOrder AS
SELECT b.Buyer_Name, o.Order_ID, o.Total_Price
FROM Buyers b
LEFT JOIN Orders o ON b.Buyer_id = o.Buyer_ID;

SELECT * FROM BuyerWithOrder

```
236 -- Tampilkan semua pembeli beserta order-nya (jika ada)
237 CREATE VIEW BuyerWithOrder AS
238 SELECT b.Buyer_Name, o.Order_ID, o.Total_Price
239 FROM Buyers b
240 LEFT JOIN Orders o ON b.Buyer_id = o.Buyer_ID;
241
242 SELECT * FROM BuyerWithOrder
```

BUYER_NAME	ORDER_ID	TOTAL_PRICE
Buyer1	O001	100
Buyer2	O002	200
Buyer3	O003	150
Buyer4	O004	220
UpdatedBuyer5	O005	80
Buyer6	O006	175
Buyer7	O007	90
Buyer8	O008	110
Buyer9	O009	130
Buyer10	O010	140

10 rows returned in 0.01 seconds [Download](#)

- b. CREATE VIEW OrderWithBuyer AS
SELECT o.Order_ID, b.Buyer_Name
FROM Orders o
RIGHT JOIN Buyers b ON o.Buyer_ID = b.Buyer_id;

SELECT * FROM OrderWithBuyer

```
244 -- Tampilkan semua order beserta buyer (jika buyer tidak ditemukan, tetap tampil)
245 CREATE VIEW OrderWithBuyer AS
246 SELECT o.Order_ID, b.Buyer_Name
247 FROM Orders o
248 RIGHT JOIN Buyers b ON o.Buyer_ID = b.Buyer_id;
249
250 SELECT * FROM OrderWithBuyer
```

ORDER_ID	BUYER_NAME
O001	Buyer1
O002	Buyer2
O003	Buyer3
O004	Buyer4
O005	UpdatedBuyer5
O006	Buyer6
O007	Buyer7
O008	Buyer8
O009	Buyer9
O010	Buyer10

10 rows returned in 0.02 seconds [Download](#)

c. CREATE VIEW OrderAndBuyers AS
 SELECT b.Buyer_Name, o.Order_ID
 FROM Buyers b
 LEFT JOIN Orders o ON b.Buyer_id = o.Buyer_ID
 UNION
 SELECT b.Buyer_Name, o.Order_ID
 FROM Buyers b
 RIGHT JOIN Orders o ON b.Buyer_id = o.Buyer_ID;

SELECT * FROM OrderAndBuyers

```

252 -- Simulasi FULL OUTER JOIN antara Orders dan Buyers
253 CREATE VIEW OrderAndBuyers AS
254 SELECT b.Buyer_Name, o.Order_ID
255 FROM Buyers b
256 LEFT JOIN Orders o ON b.Buyer_id = o.Buyer_ID
257 UNION
258 SELECT b.Buyer_Name, o.Order_ID
259 FROM Buyers b
260 RIGHT JOIN Orders o ON b.Buyer_id = o.Buyer_ID;
261
262 SELECT * FROM OrderAndBuyers
263

```

BUYER_NAME	ORDER_ID
Buyer1	O001
Buyer10	O010
Buyer2	O002
Buyer3	O003
Buyer4	O004
Buyer6	O006
Buyer7	O007
Buyer8	O008
Buyer9	O009
UpdatedBuyer5	O005

10 rows returned in 0.01 seconds [Download](#)

4. SET OPERATOR

- a. CREATE VIEW BuyerAndSeller1 AS
SELECT Buyer_Name AS Name FROM Buyers
UNION
SELECT shop_name AS Name FROM Sellers;

SELECT * FROM BuyerAndSeller1

```
263
264 -- UNION: Gabungkan nama pembeli dan nama seller (tanpa duplikat)
265 CREATE VIEW BuyerAndSeller1 AS
266 SELECT Buyer_Name AS Name FROM Buyers
267 UNION
268 SELECT shop_name AS Name FROM Sellers;
269 |
270 SELECT * FROM BuyerAndSeller1
271
```

Results Explain Describe Saved SQL History

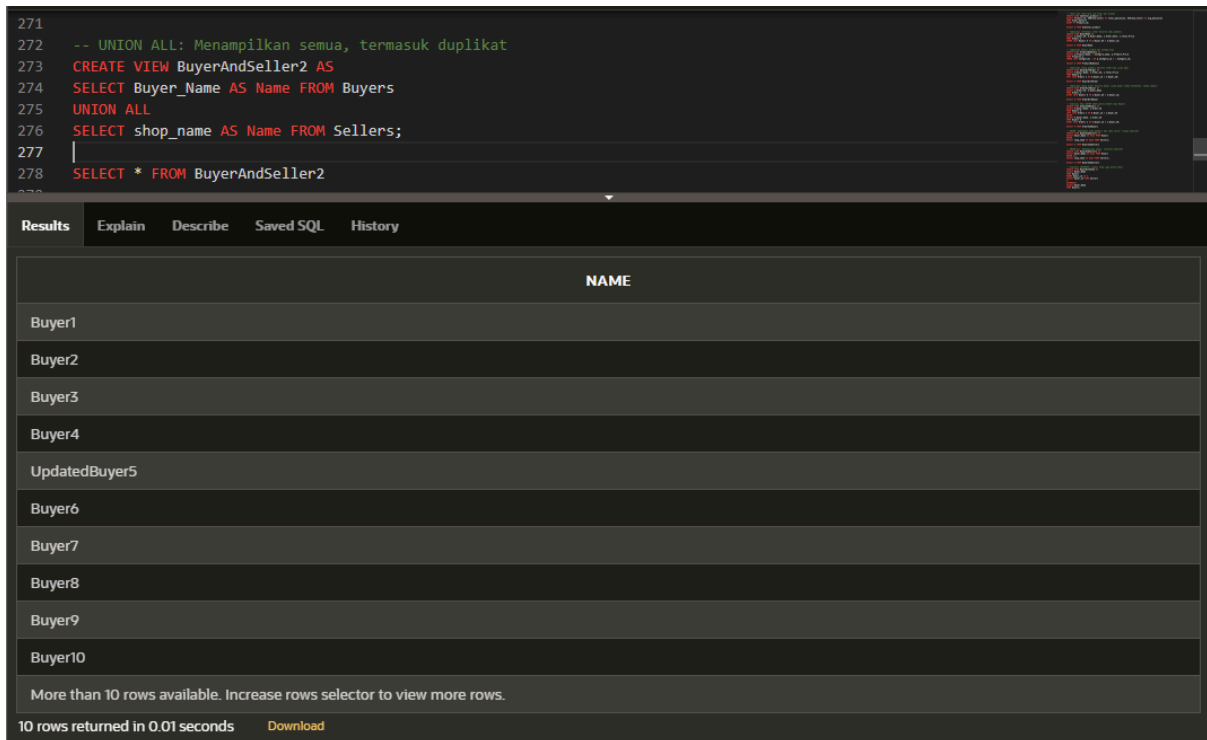
NAME
Buyer1
Buyer10
Buyer2
Buyer3
Buyer4
Buyer6
Buyer7
Buyer8
Buyer9
Shop1

More than 10 rows available. Increase rows selector to view more rows.

10 rows returned in 0.02 seconds [Download](#)

b. CREATE VIEW BuyerAndSeller2 AS
 SELECT Buyer_Name AS Name FROM Buyers
 UNION ALL
 SELECT shop_name AS Name FROM Sellers;

SELECT * FROM BuyerAndSeller2



The screenshot shows a SQL IDE interface. The top pane displays the following SQL code:

```

271
272 -- UNION ALL: Menampilkan semua, termasuk duplikat
273 CREATE VIEW BuyerAndSeller2 AS
274 SELECT Buyer_Name AS Name FROM Buyers
275 UNION ALL
276 SELECT shop_name AS Name FROM Sellers;
277
278 SELECT * FROM BuyerAndSeller2
  
```

The bottom pane, titled "Results", shows the output of the query. It includes tabs for "Results", "Explain", "Describe", "Saved SQL", and "History". The "Results" tab is active, displaying a table with one column named "NAME". The table contains 10 rows of data:

NAME
Buyer1
Buyer2
Buyer3
Buyer4
UpdatedBuyer5
Buyer6
Buyer7
Buyer8
Buyer9
Buyer10

Below the table, a message states: "More than 10 rows available. Increase rows selector to view more rows." At the bottom of the Results pane, it indicates "10 rows returned in 0.01 seconds" and provides a "Download" link.

c. CREATE VIEW BuyerWithShop AS
 SELECT Buyer_Name
 FROM Buyers
 WHERE Buyer_id IN (
 SELECT Buyer_id FROM Sellers
)
 INTERSECT
 SELECT Buyer_Name
 FROM Buyers;

SELECT * FROM BuyerWithShop

```
279
280 -- Simulasi INTERSECT (buyer yang juga punya toko)
281 CREATE VIEW BuyerWithShop AS
282 SELECT Buyer_Name
283 FROM Buyers
284 WHERE Buyer_id IN (
285 SELECT Buyer_id FROM Sellers
286 )
287 INTERSECT
288 SELECT Buyer_Name
289 FROM Buyers;
290
291 SELECT * FROM BuyerWithShop
```

Results Explain Describe Saved SQL History

BUYER_NAME
Buyer1
Buyer10
Buyer2
Buyer3
Buyer4
Buyer6
Buyer7
Buyer8
Buyer9
UpdatedBuyer5

10 rows returned in 0.02 seconds [Download](#)

d. CREATE VIEW ProductNotBeeverages AS
SELECT Product_id
FROM Order_Details

MINUS

SELECT Product_id
FROM Products
WHERE Category_id = 'C010';

SELECT * FROM ProductNotBeeverages

```

292
293 -- Kita ingin mengetahui produk yang pernah dipesan (Order_Details) tetapi tidak termasuk dalam kategori 'Beverag
294 CREATE VIEW ProductNotBeeverages AS
295 SELECT Product_id
296 FROM Order_Details
297
298 MINUS
299
300 SELECT Product_id
301 FROM Products
302 WHERE Category_id = 'C010';
303
304 SELECT * FROM ProductNotBeeverages

```

Results Explain Describe Saved SQL History

PRODUCT_ID
P001
P002
P003
P004
P005
P006
P007
P008
P009

9 rows returned in 0.02 seconds [Download](#)

5. SUBQUERY

SUBQUERY dalam WHERE

- CREATE VIEW ProductAboveAverage AS
 SELECT Product_Name, Product_Price
 FROM Products
 WHERE Product_Price > (
 SELECT AVG(Product_Price) FROM Products
);

SELECT * FROM ProductAboveAverage

```

305
306 -- Tampilkan semua produk yang harganya di atas rata-rata
307 CREATE VIEW ProductAboveAverage AS
308 SELECT Product_Name, Product_Price
309 FROM Products
310 WHERE Product_Price > (
311     SELECT AVG(Product_Price) FROM Products
312 );
313
314 SELECT * FROM ProductAboveAverage

```

Results Explain Describe Saved SQL History

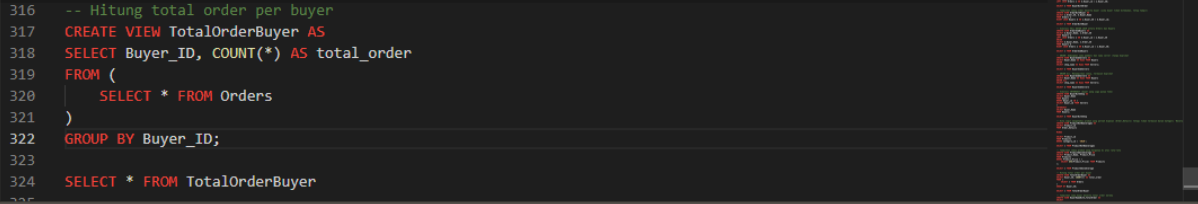
PRODUCT_NAME	PRODUCT_PRICE
Chicken	99
Tilapia	25

2 rows returned in 0.00 seconds [Download](#)

SUBQUERY dalam FROM

```
CREATE VIEW TotalOrderBuyer AS
SELECT Buyer_ID, COUNT(*) AS total_order
FROM (
    SELECT * FROM Orders
)
GROUP BY Buyer_ID;

SELECT * FROM TotalOrderBuyer
```



```
316 -- Hitung total order per buyer
317 CREATE VIEW TotalOrderBuyer AS
318 SELECT Buyer_ID, COUNT(*) AS total_order
319 FROM (
320     SELECT * FROM Orders
321 )
322 GROUP BY Buyer_ID;
323
324 SELECT * FROM TotalOrderBuyer
325
```

BUYER_ID	TOTAL_ORDER
B005	1
B007	1
B008	1
B003	1
B006	1
B009	1
B010	1
B004	1
B002	1
B001	1

10 rows returned in 0.01 seconds [Download](#)

SUBQUERY dalam SELECT

```
CREATE VIEW BuyerNameWith_TotalOrder AS
```

```
SELECT
```

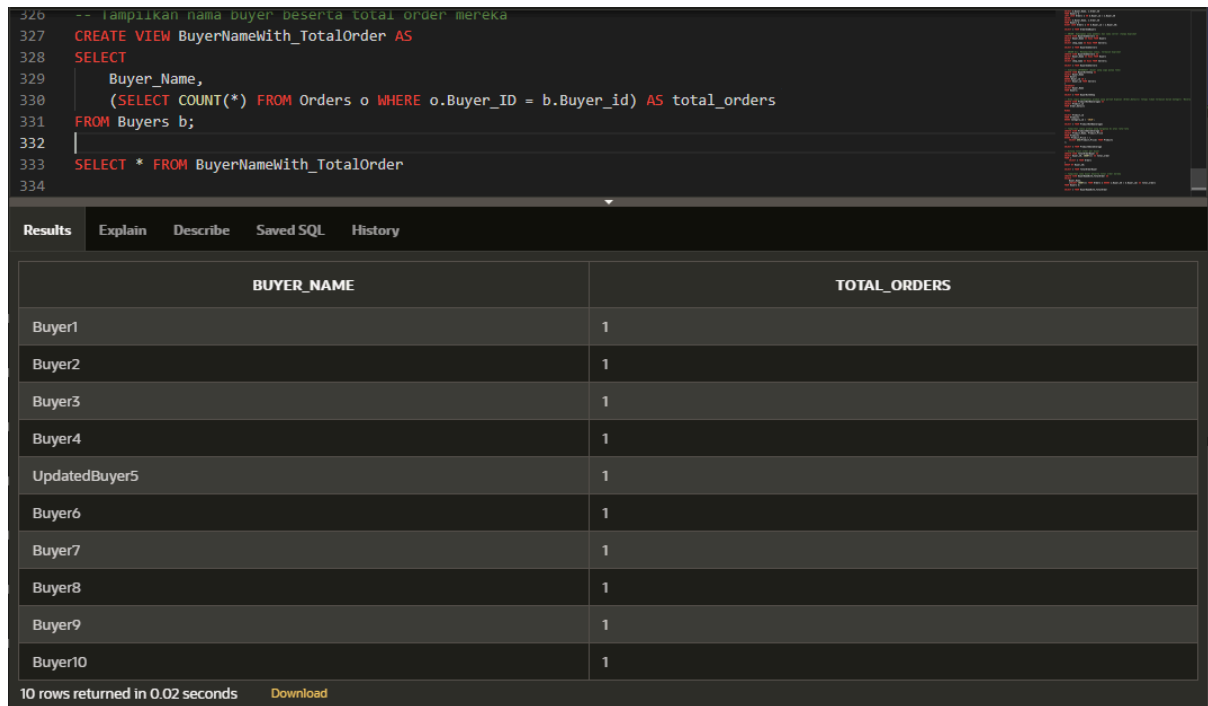
```
    Buyer_Name,
```

```
    (SELECT COUNT(*) FROM Orders o WHERE o.Buyer_ID = b.Buyer_id) AS
```

```
total_orders
```

```
FROM Buyers b;
```

```
SELECT * FROM BuyerNameWith_TotalOrder
```



The screenshot shows a SQL IDE with a script editor and a results pane. The script editor contains the following SQL code:

```
326 -- tampilkan nama buyer beserta total order mereka
327 CREATE VIEW BuyerNameWith_TotalOrder AS
328 SELECT
329     Buyer_Name,
330     (SELECT COUNT(*) FROM Orders o WHERE o.Buyer_ID = b.Buyer_id) AS total_orders
331 FROM Buyers b;
332
333 SELECT * FROM BuyerNameWith_TotalOrder
334
```

The results pane shows the output of the query, displaying 10 rows of data. The columns are **BUYER_NAME** and **TOTAL_ORDERS**. The data is as follows:

BUYER_NAME	TOTAL_ORDERS
Buyer1	1
Buyer2	1
Buyer3	1
Buyer4	1
UpdatedBuyer5	1
Buyer6	1
Buyer7	1
Buyer8	1
Buyer9	1
Buyer10	1

At the bottom of the results pane, it states "10 rows returned in 0.02 seconds" and provides a "Download" link.